

Yellow Devil

Teletransporte Distribuído

Projeto de Desenvolvimento de Sistemas Distribuídos

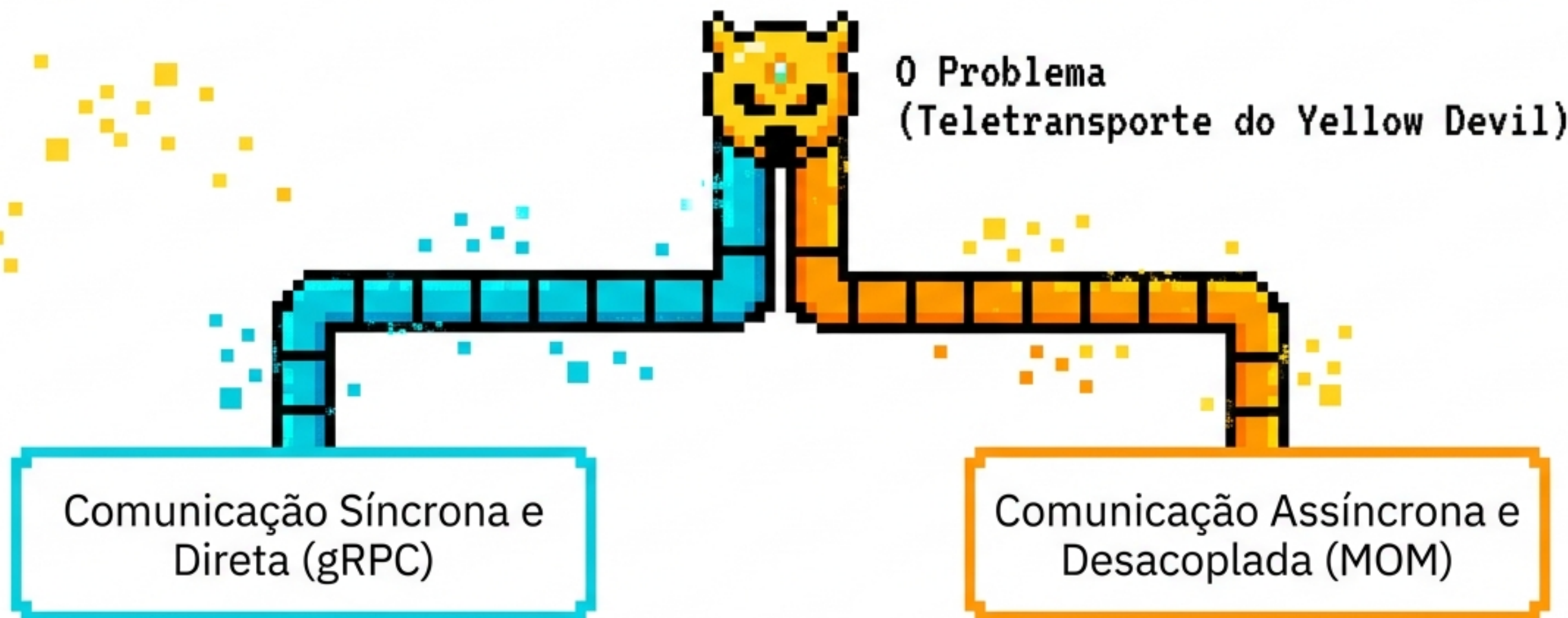
Autores: Ramon Couto Santos e Aaron Guerra Goldberg
Ano: 2026

[PRESS START]



A Missão: Um Chefe, Dois Paradigmas

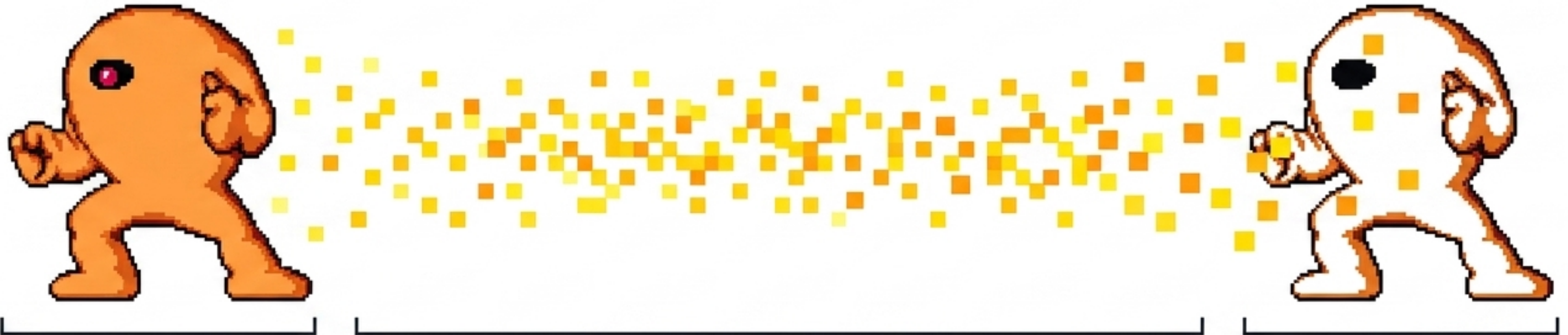
O desafio consiste em implementar a mesma mecânica de teletransporte distribuído utilizando duas tecnologias fundamentalmente diferentes.



A Metáfora do Teletransporte

Origem / Cliente

Destino / Servidor



Desintegração
(Serialização)

Transmissão de Partículas
(Mensagens)

Remontagem
(Desserialização)

Cada partícula do monstro é uma mensagem trafegando pela rede.
O chefe só é vulnerável durante esse trânsito.

Stage Select

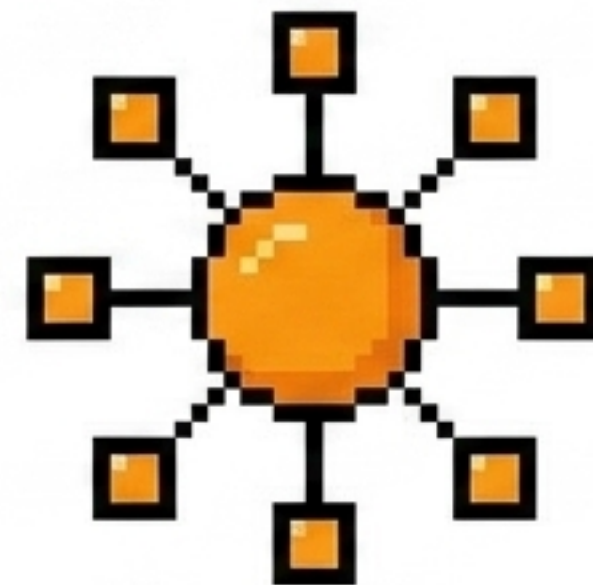
Stage 1 - gRPC

Streaming RPC



Stage 2 - MOM

Fila de Mensagens

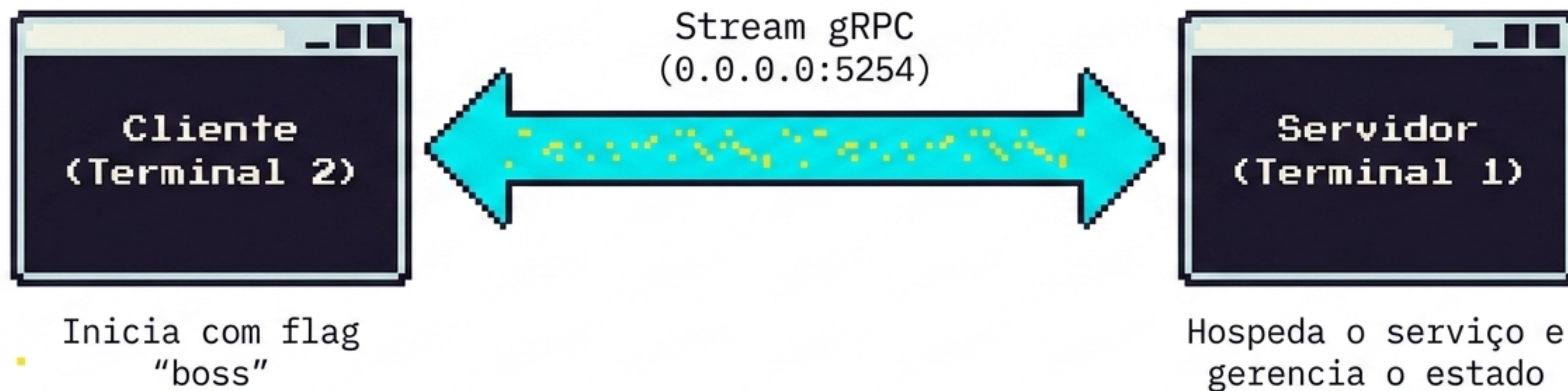


Stage 1: Implementação gRPC

Linguagem: C# (.NET 10)	Servidor: ASP.NET Core (Grpc.AspNetCore)	Cliente: Console (Grpc.Net.Client)	Contrato: Protocol Buffers (Yellow_devil.proto)
-----------------------------------	---	---	--

As partículas do boss trafegam como mensagens de um **stream** RPC contínuo. A comunicação exige uma **conexão direta, síncrona e ponto-a-ponto** entre o terminal do cliente e o servidor.

Arquitetura gRPC: Conexão Direta



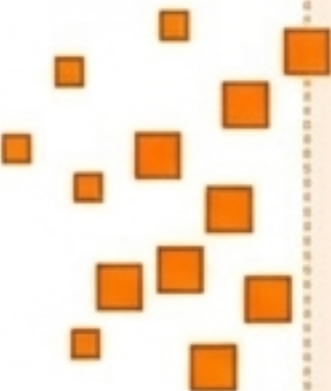
Acoplamento Espacial: O cliente precisa saber exatamente o IP do servidor para se conectar.

Stage 2: Middleware Orientado a Mensagens (MOM)

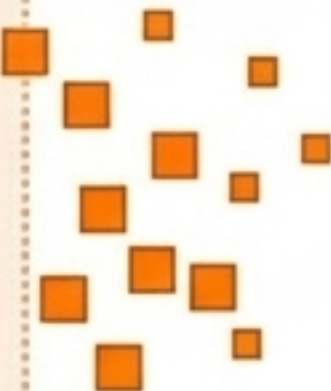
Broker:
RabbitMQ

Back-end: C#
(AMQP 5672)

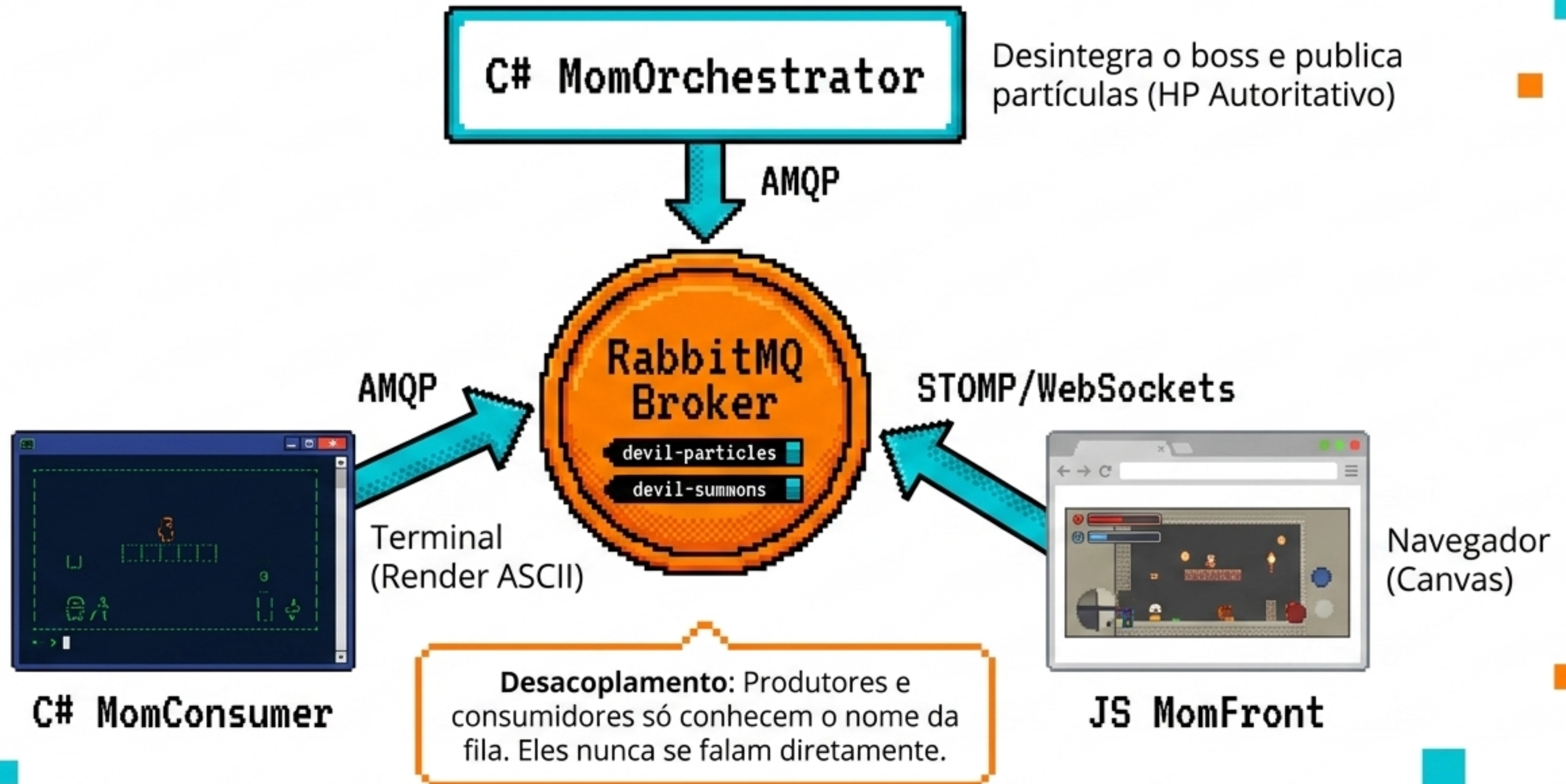
Front-end: JS Puro
(STOMP/WS 15674)



As partículas viram mensagens numa fila durável. A comunicação torna-se assíncrona, introduzindo um intermediário (Broker) que separa quem produz a mensagem de quem a consome.

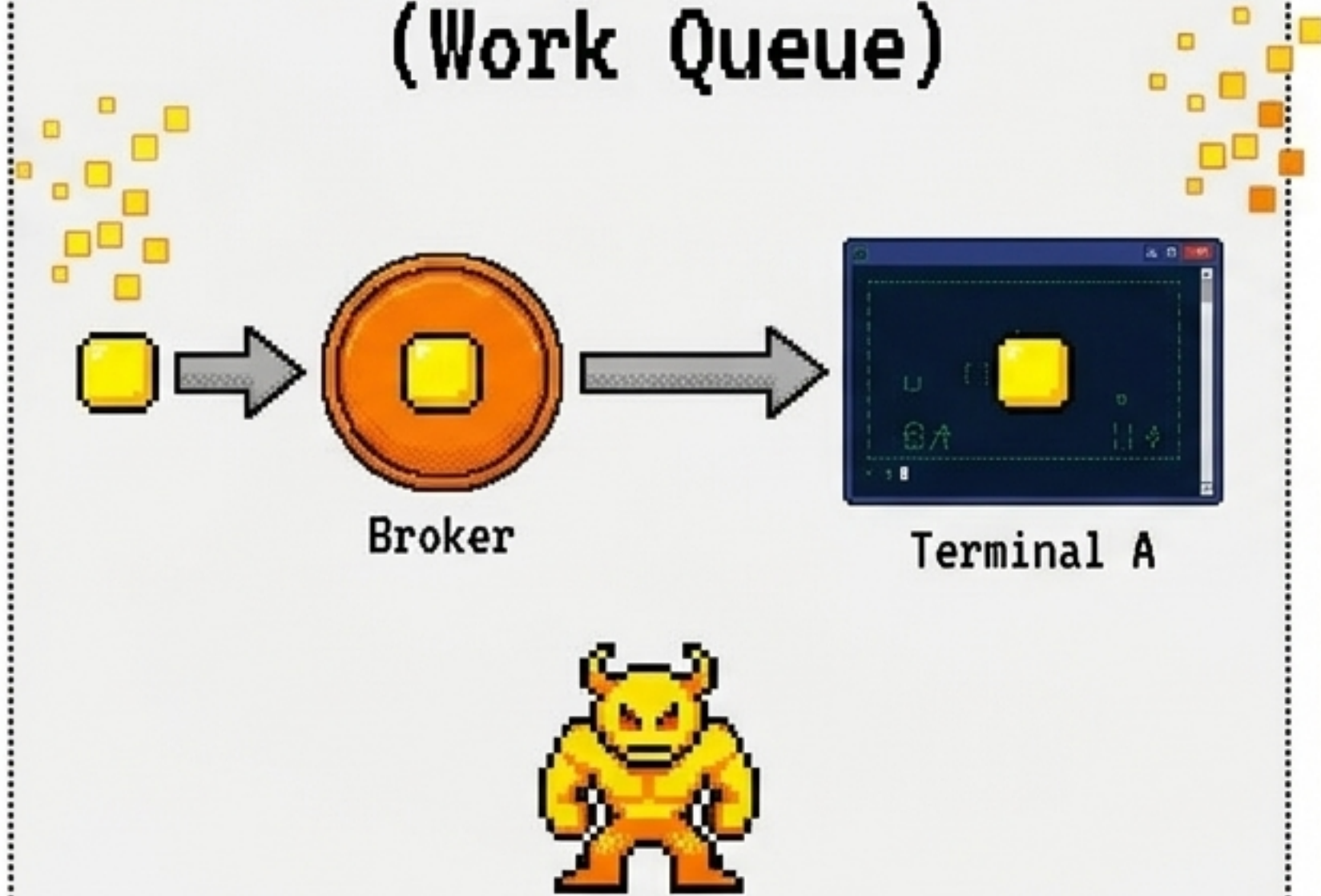


Arquitetura MOM: 0 Desacoplamento Absoluto

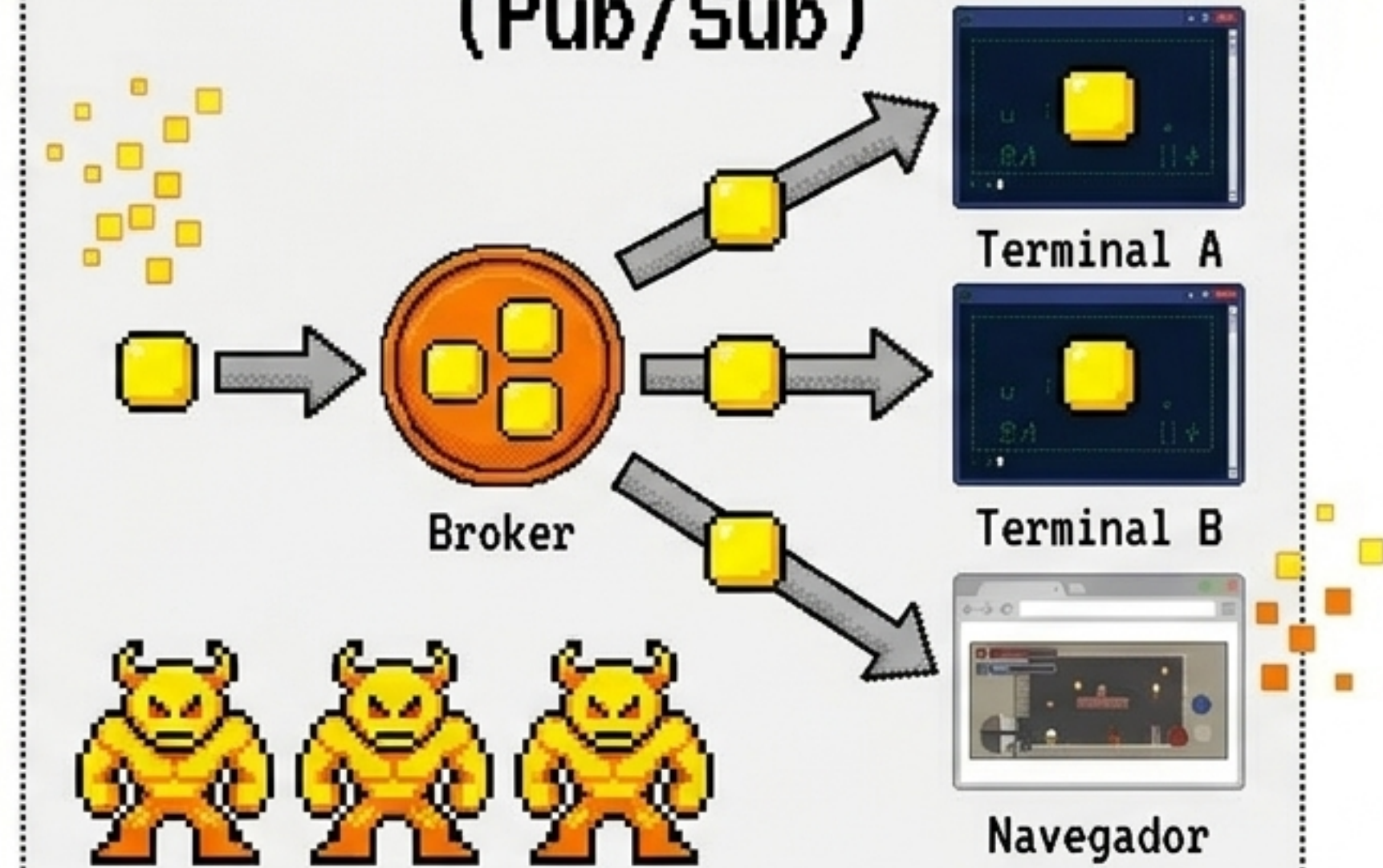


A Grande Decisão: Work Queue vs. Pub/Sub

Fila Ponto-a-Ponto (Work Queue)

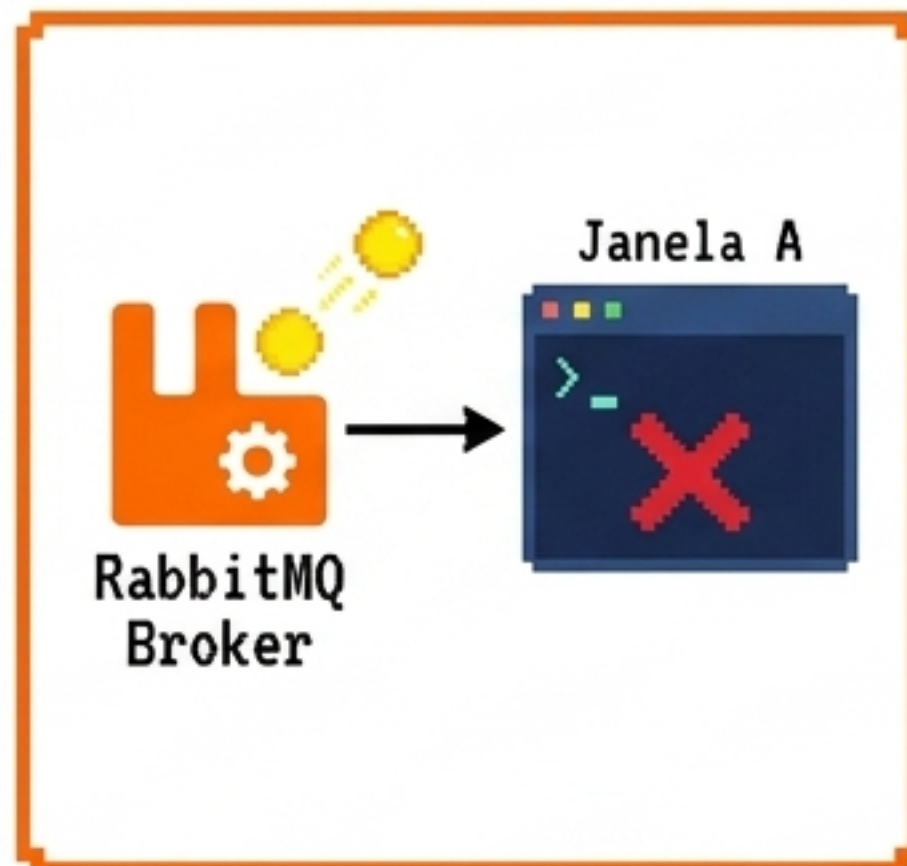


Exchange Fanout (Pub/Sub)

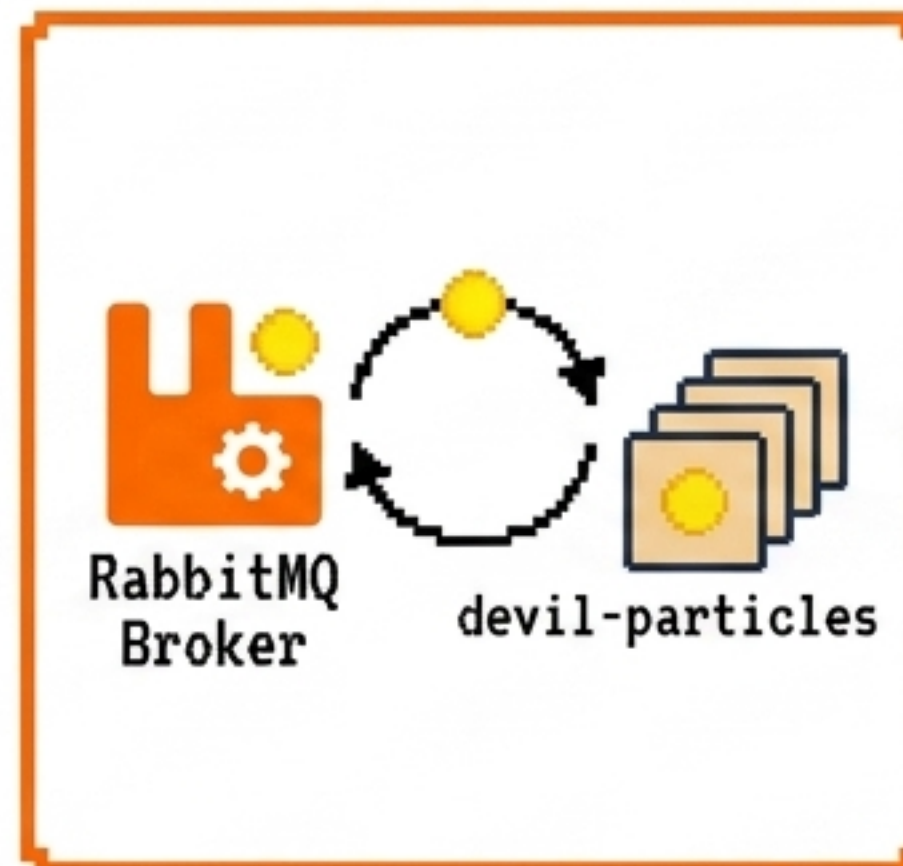


O modelo Pub/Sub geraria um broadcast, clonando o boss. A fila de trabalho garante que cada partícula seja processada exatamente uma vez.

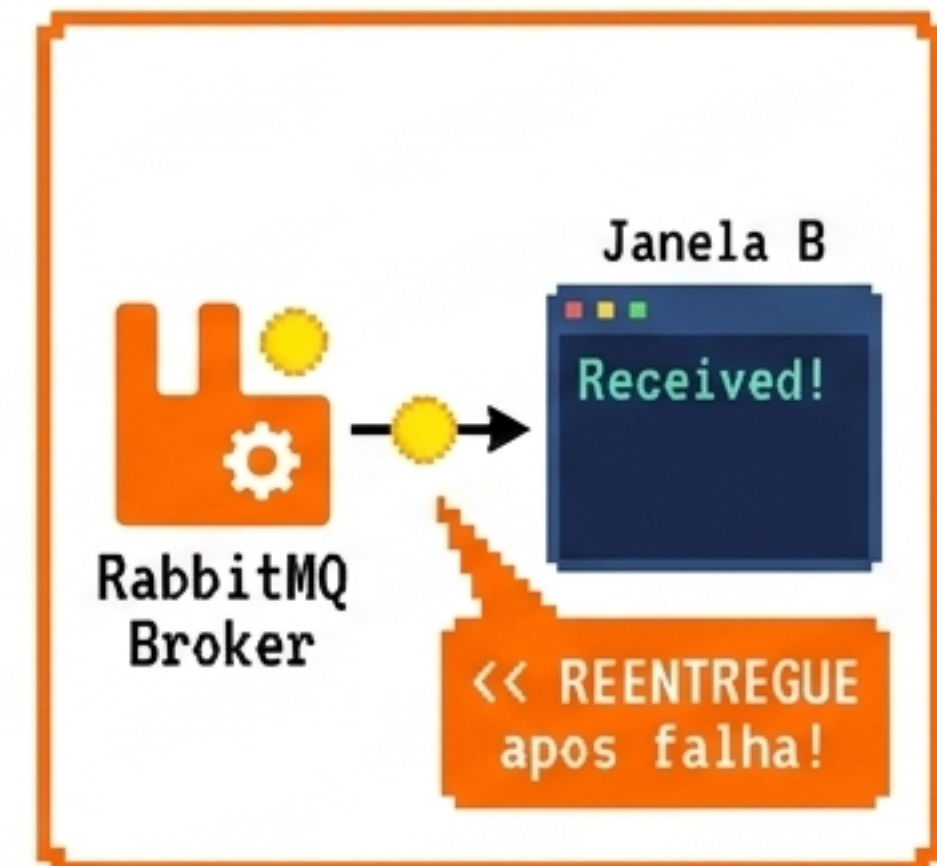
Resiliência: Garantia de Entrega com 'Ack' Manual



1. Janela A falha antes do 'ack'.



2. Broker detecta ausência de ack e devolve a partícula à fila.



3. Partícula é reentregue ao próximo consumidor disponível.

Com prefetch=1 e confirmação manual, o RabbitMQ garante que nenhuma partícula do Yellow Devil se perca no éter, mesmo se um terminal falhar no meio do teletransporte.

Modos de Jogo: Interagindo com o Sistema

Terminal C#

Controle via CLI. Requer .NET SDK.

Comandos:

- ENTER (com boss): Teleporta para o servidor.
- ENTER (sem boss): Invoca o monstro.
- sair + ENTER: Encerra o cliente.

>

Modo Sala de Aula - Navegador

← ↻ <https://monstro.local>

Acesso via Web UI (gRPC na porta 5255, MOM via STOMP).

Zero instalação, acessível via celular na mesma rede.



INVOCAR O
MONSTRO



DEVOLVER
AO SERVIDOR

Isola a complexidade distribuída atrás de uma interface simples.

Scoreboard: gRPC vs. Fila de Mensagens

Dimensão	gRPC	MOM (RabbitMQ)
Paradigma	Síncrono / Streaming Direto	Assíncrono / Baseado em Eventos
Acoplamento	Alto (Conhece o IP do Servidor)	Baixo (Conhece apenas a Fila)
Roteamento de Carga	1 para 1 (Cliente-Servidor)	1 para N (Balanceamento via Broker)
Tolerância a Falhas	Conexão cai = Stream interrompido	Mensagens aguardam na fila durável

O gRPC brilha em velocidade e comunicação direta, enquanto o RabbitMQ (MOM) oferece resiliência superior e total independência entre os nós.

Stage Clear



O teletransporte foi concluído com sucesso.
Um único conceito arquitetural, duas provas de conceito
totalmente funcionais em sistemas distribuídos.

Desenvolvimento de Sistemas Distribuídos (2026)
Ramon Couto Santos & Aaron Guerra Goldberg

[PRESS ANY KEY TO EXIT]